

Knapsackproblem in Skyrim

1st Tobias PirkI
BSC Informatik Student
JKU Linz
Linz, Österreich
k12417250@students.jku.at

2nd Michael Hinterdorfer
BSC Informatik Student
JKU Linz
Linz, Österreich
k12411630@students.jku.at

3rd Felix Anzengruber
BSC Informatik Student
JKU Linz
Linz, Österreich
k12404909@students.jku.at

4th Simon Fischer
BSC Informatik Student
JKU Linz
Linz, Österreich
k12411652@students.jku.at

5th Thiemo Ladner
BSC Informatik Student
JKU Linz
Linz, Österreich
k11708793@students.jku.at

6th Daniel Paukovits
BSC Informatik Student
JKU Linz
Linz, Österreich
k12404907@students.jku.at

Zusammenfassung—Im folgenden Paper wird das bekannte Knapsack-Problem auf das Computerspiel „The Elder Scrolls V: Skyrim“ angewandt. Dazu wird die Welt abstrahiert, die spezielle Form des Problems auf die abstrakte abgebildet und deren NP-Completeness nachgewiesen. Im Anschluss werden konkrete praktische Abläufe inklusive der Erhebung von Beispieldaten demonstriert.

Index Terms—computer science, karp reduction, SAT, video games, skyrim

I. BEGRÜNDUNG DER AUSWAHL

Beim Überlegen und suchen nach einer Projektidee ist die Gruppe auf das Knapsackproblem gestoßen, welches alle Vorgaben für das Projekt erfüllt. Nach Appell durch Frau Brandl, wie oft die gleichen Probleme gewählt und abgegeben wurden und wir kreativ werden sollten, stand für das Team schnell fest, dass das Projekt kein klassisches abschreiben eines Beweises, den es zur genüge im Internet zu finden gibt, werden sollte. Da das Videospiel „The Elder Scrolls V: Skyrim“ in der Vorlesung schon kurz bei der Angabenbesprechung erwähnt wurde und ein Teammitglied besonders begeistert von diesem ist, wurde sich, nach längerem diskutieren verschiedener Ideen, auf eine Lösung des Knapsackproblems für den Startdungeon bzw. Zelleninstanz HelgenKeep01, geeinigt.

Definitionen/Terminologie

Cells/Zellen: Das Spiel arbeitet mit vielen einzelnen Szenen, die Bethesda Cells bzw. Zellen nennt. Sie unterstützen das Videospiel hinsichtlich Performance.

RNG: Direkt als Abkürzung für random number generator, somit ein Term, der für Zufallselemente im Allgemeinen im Spiel verwendet wird.

II. ABGRENZUNG DES PROBLEMS / ABSTRAHIERUNG DER WELT

A. Einführung

Um das Knapsackproblem in einem nichtdeterministischen Videospiel anwenden zu können, müssen Teilaspekte fixiert bzw. komplett aus der Welt herausgenommen werden.

Eine erste Eingrenzung ist die Version des Spiels. Für dieses Paper wurden keine Modifikationen (als Mods weiterführend bezeichnet) verwendet. Es sei zu erwähnen, dass die Gruppe zu allen Versionen des Spieles Zugriff hatte, sich hierbei für die Datenerhebung auf das Basegame beschränkt - die später hinzugefügten DLCs für das Spiel ändern die Objekte in HelgenKeep nicht [1], daher ist die Wahl zwischen Version SSE bzw. älteren Versionen ohne DLCs trivial. Das Anniversary Upgrade bzw. die VR Version wurde hierfür allerdings nicht getestet, für diese Versionen können Unterschiede in Testläufen entstehen.

Die nächste Einschränkung der abstrahierten Welt Skyrim ist die Interior Cell HelgenKeep01. Nach dem Ende des Epilogs und der Charaktererstellung muss jeder Spieler diesen Spielbereich durchqueren und wird hinsichtlich des Open World Aspekts des Spiels bis zum Verlassen eingeschränkt. Der ganze Problemverlauf wird hier stattfinden, beginnend mit dem Eintreten in den Turm und endend mit dem Verlassen nach Himmelsrand nach dem Durchqueren der anliegenden Höhle. Weitere wichtige Aspekte werden in den folgenden Unterkapitel noch ausführlicher beschrieben.

B. Spielerlevel

Da der Charakterlevel über die Ausdauer die Tragkraft bestimmt, wird von einem Charakter ausgegangen, der entweder:

- Im Spielerlevel aufsteigen kann, aber nie das Attribut Ausdauer wählt und somit keine erhöhte Tragfähigkeit bekommt.
 - In einem normalen Spielverlauf sind ein paar Levelaufstiege vor dem Verlassen der Zelle möglich.
 - Besonders hervorzuheben ist, dass es durch das Ausnutzen verschiedener Spielmechaniken (Glitches) möglich ist, schon vor Verlassen der Zelle in einer unendlich langen Schleife zu landen und theoretisch unbegrenzt im Level aufzusteigen.
- Nicht im Level aufsteigt. Somit ist eine konstante Ausdauer gegeben.

C. Tragkraft

Ein Spieler startet mit einer Tragkraft von 300 [2] und kann diese um 5 Punkte pro Levelaufstieg und Wahl von Ausdauer (anstatt Magicka bzw. Gesundheit [3]) erhöhen. Durch Ausschluss von Levelaufstieg bzw. Wahl des Attributes Ausdauer ist somit in diesem Sinne schon eine konstante Tragkraft gegeben.

Im Spiel existieren auch weitere Möglichkeiten, die Tragkraft zu erhöhen, wie z.B. Tränke, verzauberte Ausrüstung, Magie, uvm. Diese Strategien sind zwar nicht umsetzbar vor dem Verlassen der Zelle „HelgenKeep01“, werden zur Sicherheit für die Verifikation allerdings trotzdem herausgenommen.

D. Objekte

Um die Objekte, die Skyrim intern verbaut hat klar unterscheiden zu können, bedarf es einigen Definitionen:

- Objekt - die Menge aller abrufbaren Daten, die die Welt darstellen.
- Items - Objekte, die der Spieler aufsammeln und, wenn möglich, eine Interaktion hervorrufen kann. Sie können physischer Bestandteil der Welt sein, jedoch auch rein logisch im Inventar des Spielers existieren. $\text{Items} \subset \text{Objekte}$
- Strukturen - Objekte, die die Welt darstellen und mit denen der Spieler physikbasiert interagieren kann. Diese müssen nicht Rigid Bodys enthalten, müssen aber einen leeren dreidimensionalen Raum mit äquivalenten Volumen vollständig füllen. $\text{Strukturen} \subset \text{Objekte} \wedge V_s = V_e | V_e \in \emptyset \subset \mathbb{R}^3$
- Container - Elternobjekte, die Referenzen für Kinderobjekte enthalten. Diese Kinderobjekte sind immer Items. Diese können statisch (Kisten) und dynamisch (NPCs) sein. $\text{Container} \subset \text{Objekte}$

Intern unterscheidet Skyrim Objekte mit Hilfe von Recordtypen [4], die zuvor definierten Objektgruppen können mit Hilfe dieser klassifiziert werden:

- Container: FLOR, CONT, ACHR
- Items: ARMO, WEAP, AMMO, ALCH, INGR, MISC, BOOK, LVLI (= leveled items)
- Strukturen: nicht relevant, daher nicht aufgelistet (z.B. REFR)

E. Actors

Da Rüstungen und Waffen einen großen Teil der verfügbaren Items in Helgen entsprechen, wurde entschieden, NPCs bzw. Actors nicht aus der Welt zu nehmen. Es ist zwar möglich, zu Beginn Hadvar oder Ralof zu folgen und so jeweils die andere Fraktion (Sturmmäntel bzw. kaiserliche Soldaten) zu bekämpfen, allerdings ist es auch möglich, die Ausrüstung der Verbündeten mit Hilfe der Taschendiebstahlfähigkeit zu stehlen.

Wichtig bei den Fraktionen sind vor allem die Unterschiede der Rüstungssets. In der Zelle existieren für kaiserliche Soldaten Offiziere und Soldaten. Die Offiziere tragen unter anderem auch schwere Rüstung, die Soldaten nur leichte.

Sturmmäntelsoldaten tragen ausschließlich leichte Rüstung. Ohne Taschendiebstahlfähigkeit könnte man also im Fall, dass nicht alle Verbündeten sterben, bei einer der beiden Route bessere Resultate erzielen.

F. RNG

Um nun das Spiel so deterministisch wie möglich zu gestalten, müssen die Container, die zufallsbasiert arbeiten getrennt von den anderen Objekten erhoben werden (mit einer der Gründe für die getrennte Definition).

Skyrim besitzt Zufallselemente, um jeden Spielverlauf anders zu gestalten. Für das Projekt relevant sind:

- Leveled Lists: Eine Liste mit Items, die angibt, zu welcher Wahrscheinlichkeit ein Spieler eine gewisse Anzahl n Items erhält. Starken Einfluss darauf hat der Spielerlevel und die Encounterzone.
- Leveled Items: Items, die in verschiedenen Stufen bzw. Qualitäten existieren und abhängig von Spielerlevel bzw. Encounterzone angepasst werden. In Helgen existieren nur drei Potions, die wir auf die unterste Stufe anpassen.

[5]

Um die Container nun so anzupassen, dass RNG nicht mehr existiert, wurde entschieden, alle Container einzeln anzusehen und in unserer abstrahierten Welt nur Items in Containern zuzulassen, die das Spiel mit Sicherheit generiert (Ausnahmen siehe nächstes Kapitel).

G. Weitere mögliche Inkonsistenzen

Kartoffeln: Items der Containergruppe MiscSack* wurde ein Wert von einer Kartoffel zugewiesen. Da diese eine große zufällig generierte Auswahl an Items aus einem Set von Obst, Gemüse und Salz generieren, wurde entschieden, diese im Vorhinein mit dem Item des schlechtesten Werts zu fixieren: einer Kartoffel, Wert 1 Gewicht 0.1 [6]. Im Verifizierungsprozess werden diese bei anderer Generierung angerechnet.

Hadvar und Ralof: Da sich das Inventar der beiden unterscheidet, werden diese aus der Welt herausgenommen. Taschendiebstahl ist bei diesen NPCs nicht zulässig.

Soldaten - Sturmmäntel und Kaiserliche

Die Kaiserlichen Soldaten sind in ihrer Waffen und Rüstungsausstattung einheitlich und deterministisch gehalten - bei den Sturmmänteln wird die Ausstattung etwas anders gehandhabt. Vor allem die Waffen werden größtenteils zufallsgeneriert zugewiesen. Alle Rüstungs- und Waffenitems der Soldaten werden aus unserer Welt herausgenommen.

Banditen

Einige Zeit nach der Flucht von Helgen wird die Stadt von Banditen besiedelt. Da diese zwar nicht vor beenden der Questline spawnen, allerdings in der gleichen Zelle gespeichert werden, werden diese hiermit herausabstrahiert.

III. KNAPSACK PROBLEM

A. Problembeschreibung - Intuitiv

Man stelle sich einen Rucksack vor, der räumlich betrachtet ein unbegrenztes Fassungsvermögen aufweist. Nun möchte man mit diesem Rucksack ausgestattet aus einem Gebiet des Spiels Skyrim den höchstmöglichen Wert an Gegenständen erbeuten. Da der Rucksack unendlich viel Platz aufweist, ist man natürlich dazu verleitet einfach alles mitzunehmen was da ist, allerdings fällt einem auf, dass man selbst nicht unbegrenzt viel Gewicht tragen kann. Nun steht der Spieler also vor der Herausforderung zu entscheiden, welche Gegenstände er mitnehmen und welche er zurücklassen möchte, um seinen Gewinn unter der Einschränkung zu maximieren.

B. Problembeschreibung - Formell

Sei G eine endliche Menge bestehend aus N 2-Tupeln in der Form $G = \{(v_i, w_i) \mid i \in [0, N]; v_i, w_i \in \mathbb{R}^+\}$, wobei v_i dabei den Wert und w_i das Gewicht des Gegenstandes G_i im Spiel beschreiben. Nun sei $W \in \mathbb{R}^+$ das maximale Gewicht das der Spieler tragen kann. (Im Folgenden wird W auch als Kapazität des Rucksacks oder des Inventars betrachtet) Das Problem vor dem der Spieler steht ist es ein $G_W \subseteq G$ zu finden, sodass $\sum_{i=0}^{|G_W|} v_i$ maximal wird, während $\sum_{i=0}^{|G_W|} w_i \leq W$ gilt. [vgl. 7]

Da für diese Arbeit allerdings ein Entscheidungsproblem - also ein Problem das mit „Ja“ oder „Nein“ beantwortet werden kann - in Form einer Sprache benötigt wird, sei diese wie folgt definiert:

$$\begin{aligned} \text{PACK} := \{ \perp(G_W, W, V) : \\ G_W \subseteq G, W, V \in \mathbb{R} \\ \wedge \left(\sum_{i=0}^{|G_W|} v_i \geq V \right) \\ \wedge \left(\sum_{i=0}^{|G_W|} w_i \leq W \right) \} \end{aligned}$$

Ein Bitstring ist also genau dann in der Sprache PACK, wenn G_W eine Untermenge von G ist, sodass die Summe der Gewichte in G_W W nicht übersteigt und die Summe der Werte der Gegenstände in G_W den Zielwert V übersteigt.

IV. ARGUMENTATION FÜR NP-COMPLETENESS

Da nun dank Abschnitt II-A Einführung klar ist, welches Subset beziehungsweise welcher Teil der Spielwelt von Skyrim in dieser Arbeit behandelt wird, und dank Kapitel III Knapsack Problem auch bekannt ist, wie dieses Problem aussieht und sich verhält, kann nun in den folgenden Passagen auf die Problemklasse NP-Complete geprüft werden. Dafür sind laut der Definition 9.8 im Skriptum der Vorlesung Berechenbarkeit und Komplexität zwei Eigenschaften zu erfüllen:

- NP-Membership: Eine Sprache die ein Entscheidungsproblem darstellt ist in NP, wenn es keinen effizienten Algorithmus zur Lösung gibt, aber eine effiziente Methode zur Validierung einer potenziellen Lösung.

- NP-Hardness: Ein Entscheidungsproblem ist NP-Hard, wenn es mindestens so schwer ist, wie alle anderen Sprachen in NP. Formell geschrieben bedeutet das: $(\forall L \in \text{NP} : L \leq_p A) \implies A \in \text{NP-HARD}$

[vgl. 8, S. 102, 84] Um also nun zu beweisen, dass die hier behandelte spezielle Auslegung des Knapsack Problems NP-Complete ist, wird zuerst gezeigt, dass das allgemeine Knapsack Problem die Bedingungen erfüllt. Darauf aufbauend lässt sich dann feststellen, dass auch das Sammeln von Items in Skyrim Dungeons mit begrenzter Kapazität des Inventars des Spielers unter den gegebenen Bedingungen laut II-A Einführung NP-Complete ist.

A. NP-Membership

Damit ein Entscheidungsproblem in Form einer Sprache in der Komplexitätsklasse NP liegt, muss ein Zertifikat $y \in \{0, 1\}^*$ mit einer Turing Machine M in polynomieller Zeit verifizierbar sein.[8]

a) *Zertifikat*: Als Zertifikat wählen wir einen Bitvektor der gewählten Teilmenge G_W :

$$\begin{aligned} N &= |G| \\ y \in \{0, 1\}^N, y_i &= 1 \Leftrightarrow (v_i, w_i) \in G_W \end{aligned}$$

also N Bits. Das Zertifikat ist damit $O(N)$ und somit polynomiell in der Eingabegröße N .

b) *Verifizierer*: Eingabe: Instanz (G, W, V) und Zertifikat y .

- Beginne mit $S_w = 0, S_v = 0$
- Schleife mit i von $0 \dots N$
 - falls $y_i = 1$: addiere $S_w = S_w + w_i, S_v = S_v + v_i$
- Accept genau dann, wenn $S_w \leq W \wedge S_v \geq V$

c) *Laufzeit*: Schleife mit genau N durchläufen, wobei jeder Durchlauf nur eine lineare Anzahl an Vergleichen und Additionen (abhängig nur von N) durchführt. Somit ist die gesamte Laufzeit auch polynomiell.

d) *Folgerung*: $\text{PACK} \in \text{NP}$

□

B. NP-Hardness

Um zu zeigen, dass $\text{PACK} \in \text{NP-HARD}$ reicht es eine Karp-Reduction durchzuführen. Dafür ist eine Möglichkeit zu finden 3-SAT (ein bekanntes NP-COMplete Problem) mittels PACK darzustellen. Um eine 3-SAT Formel als Knapsack Problem darstellen zu können, ist eine Transformation von Clauses und Literals zu Items zu finden. Außerdem ist festzulegen, wie der Wahrheitswert mit der Kombination Gewicht (W) und Wert (V) zusammenhängt.

1) *Abbildung zwischen 3-SAT und PACK*:

a) *Variablen*: In einer beliebig langen 3-SAT Formel gibt es eine gewisse Anzahl $X \in \mathbb{N}$ an Variablen x . Diese können zusätzlich noch als $\neg x$ auftreten, bleiben jedoch dieselbe Variable. Seien nun $x_0 \dots x_{X-1}$ die verwendeten Variablen in der 3-SAT Formel, dann werden diese vorerst so dargestellt:

$$x_i = (\text{CAT}_{j=0}^i 0 \text{ CAT } 1 \text{ CAT}_{i+1}^X)_{10}$$

[vgl. 9] Bei einer Formel mit 6 Variablen würden das die Repräsentationen $x_0 = 100000_{10}$, $x_1 = 010000_{10}$, $x_2 = 001000_{10}$, ... und $x_5 = 000001_{10}$ sein. Für X Variablen ergeben sich also Zahlen in der Basis 10 die positionsweise durchnummeriert sind.

b) *Clauses*: Eine Clause in einer 3-SAT Formel besteht aus 3 Literalen aus $\{x_0 \dots x_{X-1}\} \cup \{\neg x_0 \dots \neg x_{X-1}\}$ verknüpft mit dem $\vee$ -Operator. Somit muss eine der Literalen in der Clause 1 sein, um die gesamte Clause auf 1 zu stellen. Um daraus einen mit den Variablen summierbaren Ausdruck (Anforderung für PACK) zu machen, wird die Teilnahme an einer Clause pro Variable vermerkt. Zusätzlich wird dies auch für jede Negation gemacht, da es jetzt einen Unterschied macht, ob die Variable selbst, oder die Negation in einer Clause ist.

Sei $C \in \mathbb{N}$ die Anzahl der Clauses, dann wird jede Variable dupliziert und wie folgt behandelt:

$$x_i = x_i * 10^C \text{ CAT } \text{CAT}_{j=0}^C \begin{cases} 1, & x_i \in \text{Clause}_j \\ 0, & x_i \notin \text{Clause}_j \end{cases}$$

$$\neg x_i = x_i * 10^C \text{ CAT } \text{CAT}_{j=0}^C \begin{cases} 1, & \neg x_i \in \text{Clause}_j \\ 0, & \neg x_i \notin \text{Clause}_j \end{cases}$$

[vgl. 9] Somit ergeben sich $2X$ Variablen mit einer Länge von $X + C$.

Zusätzlich dazu muss noch ermöglicht werden, dass ein einziges wahres Literal eine ganze Clause wahr macht. Hierfür fügt man in die Menge G die hinter PACK steht noch Platzhalter ein. Da eine Clause aus drei Literalen besteht, aber nur eine wahr sein muss, werden tatsächlich zwei Platzhalter benötigt. Diese werden wie folgt konstruiert:

$$p_{c_i} = \text{CAT}_{j=0}^X 0 \text{ CAT } \text{CAT}_{j=0}^i 0 \text{ CAT } 1 \text{ CAT}_{j=i+1}^C 0$$

[vgl. 9]

c) *Zielgewicht und Zielwert*: Für das Entscheidungsproblem PACK sind nun noch die Gewichts- und die Wertgrenzen zu bestimmen. Der Einfachheit halber seien die beiden gleich ($V = W$), wodurch nur noch ein Wert zu definieren ist. [vgl. 9] Dieser muss drei Bedingungen bündeln:

- Summiert man die Gegenstände in G_W so muss die gleichzeitige Verwendung von x_i und $\neg x_i$ immer zu einem zu hohen Gewicht führen. Das verhindert, dass man einer Variable von 3-SAT zugleich 0 und 1 zuweist.
- Lässt man eine Variable undefiniert muss immer ein zu niedriger Wert herauskommen.
- Eine Clause muss immer denselben Wert haben um validierbar zu sein. Am einfachsten ist es, $V = W$ wie folgt zu definieren:

$$V = W = \text{CAT}_{j=0}^X 1 \text{ CAT } \text{CAT}_{j=0}^C 3$$

Für eine 3-SAT Formel mit 6 Variablen und 3 Clauses wäre also die Zahl 111111333 der Zielwert beziehungsweise das Zielgewicht ($V = W$).

2) *Beispiel*: Man betrachte die 3-SAT Formel $\varphi = (a \vee b \vee c) \wedge (d \vee \neg c \vee \neg a)$. Die Darstellung dieser Formel über PACK ist die folgende (Achtung: $V = W \implies v_i = w_i$):

$$G = \{100010, 100001, \\ 010010, 010000, \\ 001010, 001001, \\ 000101, 000100, \\ 000010, 000010, \\ 000001, 000001\}$$

$$V = W = 111133$$

Verwendet man das konkrete satisfying Assignment 1001 und summiert die ersten vier Stellen auf so erhält man 1111 somit ist schon mal der erste Teil der Lösung korrekt. Nun summiert man die letzten beiden Stellen auf und erhält nur 22, was nicht schlimm ist, da genau dafür die Platzhalter da sind. Man verwendet also von beiden Platzhalterpaaren genau einen und erhält die Lösung 111133, was genau das gesuchte Gewicht ist.

Ein Beispiel für ein unsatisfying Assignment wäre 1110. Hier bekommt man in den letzten beiden Stellen 30 was mit keinem Platzhalter in G auf 33 gebracht werden kann.

3) $\varphi_{3\text{-SAT}}$ ist sat. $\iff$ PACK hat eine Lösung:

a) $\implies$: Hat man ein Satisfying Assignment gefunden, so wählt man für jede Variable den richtigen Wert aus G , also entweder x_i oder $\neg x_i$ und summiert diese auf. Zum Schluss versucht man die Stellen der Clauses mit den entsprechenden Platzhaltern in Richtung 3 aufzuwerten, was gelingt da ja mindestens eine der drei Variablen der Clause bereits 1 beigetragen hat. [vgl. 9]

b) $\impliedby$: Hat man eine Auswahl an Gegenständen die zum gewünschten Wert führen vorliegen, bestätigen die ersten X Stellen, dass jede Variable genau einen fixen Wert zugewiesen bekommen hat. Die letzten C Stellen bestätigen dann, dass mindestens eine der drei Variablen jeder Clause 1 beigetragen hat, da die Platzhalter lediglich 2 beitragen können. Somit ist sichergestellt, dass auch diese Richtung zu einer Satisfying Konfiguration führt. [vgl. 9]

Damit ist gezeigt, dass eine eingeschränkte Form von PACK mindestens so kompliziert ist wie 3-SAT, wodurch auch sicher allgemein gilt

$$3\text{-SAT} \leq_p \text{PACK}$$

und

$$\text{PACK} \in \text{NP-HARD}$$

gilt. □

Angemerkt sei dabei noch, dass durch die Gleichsetzung $V = W$ Spezifika von Skyrim verlorengehen. Das ist aber weder für die Komplexitätseinordnung (wäre nur noch schwieriger), noch für die Relation zum Spiel (ist ja nur ein Spezialfall) von Bedeutung.

C. NP-Completeness

Kurzesagt:

$$\begin{aligned} \text{PACK} &\in \text{NP} \wedge \text{PACK} \in \text{NP-HARD} \\ \therefore \text{PACK} &\in \text{NP-COMplete} \end{aligned}$$

Da in den vorangegangenen Abschnitten IV-A NP-Membership und IV-B NP-Hardness gezeigt wurde, dass PACK tatsächlich sowohl in NP, als auch in NP-HARD liegt, und dies die Anforderungen an ein NP-COMplete Entscheidungsproblem sind ([siehe 8]), ist die Folgerung $\text{PACK} \in \text{NP-COMplete}$ wahr für das mathematische Problem, sowie für die Anwendung in Skyrim oder anderen Spielen mit vergleichbarer Logik.

V. DATENSET UND DATENERHEBUNG

SSEEdit

Auf der Suche nach einem Tool, dass es ermöglicht, Skyrim auf einer tieferen Ebene anzusehen, stach SSEEdit heraus [10]. SSEEdit ermöglicht seinen Nutzern die genauere Analyse und Einblick in eine „Low-Level“ Ansicht des Videospiels. Das Tool wurde auch für viele weitere Bethesda Titel entwickelt, gemeinsam werden diese in der Modding Community als xEdit bezeichnet. [10] Die Daten werden in einer Parent-Child Beziehung dargestellt. Die einzelnen Spielbereiche, die durch Labildschirme getrennt sind, nennt Bethesda Cells, uns interessiert die Interior Cell HelgenKeep01. HelgenKeep01 enthält alle Objekte des für uns relevanten Spielbereichs, zu finden ist diese unter dem Pfad *Cell/Block8/Sub-Block4/HelgenKeep01*, wobei dieses zwei Kindelemente besitzt, in temporary werden alle Objekte der Welt gespeichert, mit denen der Spieler interagiert, in persistent logische Objekte, die für die interne Spiellogik wichtig sind. Angefügt ist ein Beispiel der Ansicht von SSEEdit: 

Creation Kit

Ein wichtiges Tool für das genauere Ansehen der Elemente ist das Creation Kit. Über SSEEdit ist es schwierig, die Leveled Lists genauer zu bestimmen und in die Inventare der NPCs zu sehen. Das Creation Kit hilft hier ab und liefert auch die genauen Positionen der Objekte im Spiel. Dies erleichtert auch das finden von unklar benannten Objekten (z.B. DummyPotion). In der angehängten Datei ist ein Beispiel einer Ansicht des Creation Kits sowie die gerenderte Zelle HelgenKeep01: 

„The Elder Scrolls V: Skyrim“

Für das Verifizieren der Erhebung ist eine Kopie des Videospiels notwendig (zumindest die Standardedition). Wie am Anfang erwähnt, wurde das Paper mit Hilfe der Standard Edition geschrieben und die zugehörige abstrahierte Welt auf dieser aufgebaut.

Datenerhebung

Grob können die Objekte in drei Gruppen eingegrenzt werden:

- Strukturen
- Container
- Items

Für das Knapsack Problem sind nur Container und Items wichtig - die Definitionen aus Kapitel II-A werden auch hier verwendet. Die Datenerhebung erfolgte unter strenger Einhaltung der im II-A erwähnten Regeln und Abstrahierung, eine Invalidierung des Verifiers ist nicht möglich. Die Daten wurden mit Hilfe mehrerer Pascal Skripte aus SSEEdit als CSV Daten eingeholt, unterteilt in die Untergruppen:

- Items
- Container
- Actor

Die Gruppe Actor sollte ursprünglich Teil der Container CSV sein, leider funktionieren die Objektgruppen praktisch grundlegend verschieden. Mit Hilfe des Creation Kits wurde nach einer ersten Erhebung auf die Container und Actor tiefer zugegriffen und inspiziert, welche Items von RNG betroffen und welche zulässig sind.

Datenset

Das Datenset ist mitsamt allen wichtigen Skripten auf dem GitHub des Projekts hier zu finden. Die Daten wurden aus der Vanilla Version erhoben und können mit Hilfe von SSEEdit & einer Installation des Spiels reproduziert werden.

Disclaimer: Gewicht und Wert der Gegenstände wurden mit Hilfe des USP Wiki händisch eingeholt.

VI. VERIFIZIERUNGSPROZESS PRAKTISCH

Um nun das Ergebnis unserer Forschung praktisch im Videospiel zu verifizieren, ist es hinreichend, einen Spielverlauf zu durchlaufen. Wichtig sei zu erwähnen, dass zuvor fixierte Variablen, die auf Zufall aufbauen, getrennt im Lauf behandelt werden müssen (Items aus dem Container TreasBanditChest-Boss, Items aus den Containern MiscSack*, erwähnt in II-A).

VII. PRAKTISCHE IMPLEMENTIERUNG

Um das Rucksackproblem unter realitätsnahen Bedingungen zu simulieren, wurde eine Java-Applikation entwickelt, die das Inventarsystem des Spiels *The Elder Scrolls V: Skyrim* als Datengrundlage nutzt. Ziel des Algorithmus ist es, aus einer Liste von Beutestücken (Items) eine Kombination zu finden, die einen definierten Goldwert-Schwellenwert erreicht, ohne das maximal zulässige Tragegewicht des Spielers zu überschreiten. Der hier beschriebene Code kann in unserem Github-Repository eingesehen werden. [siehe. 11]

A. Modellierung der Spielobjekte

Die Datenstruktur wurde spezifisch auf die Spielmechanik zugeschnitten:

- Item (Record): Speichert die aus den Spieldateien extrahierten Attribute: Name (z.B. „Dragon Bone“), Gewicht (float) und Goldwert (int).

- **Knapsack-Klasse:** Repräsentiert das Inventar des Spielers. Sie aggregiert dynamisch das aktuelle Gesamtgewicht und den Gesamtwert. Ein Status-Enum (SUCCEEDED, FAILED, IN_PROGRESS) gibt an, ob die aktuelle Kombination Constraints erfüllt, noch nicht fertig berechnet oder nicht erfüllt.

B. Algorithmischer Ansatz: Parallelisierter Brute-Force

a) *Rekursion und Speicheroptimierung:* Der Kern besteht aus einer rekursiven Tiefensuche. Ein wesentlicher Aspekt der Implementierung ist die Vermeidung von OutOfMemory-Fehlern. In einer frühen Version wurden bei jedem Rekursionsschritt Kopien der gesamten Item-Listen erstellt. Dies wurde zugunsten einer speichereffizienten Methode korrigiert: Der Algorithmus arbeitet nun auf einer globalen Liste. Ob ein Item bereits im „Rucksack“ liegt, wird durch einen Referenzvergleich geprüft. Dies spart signifikant Heap-Speicher, da lediglich die Referenzen der Skyrim-Items auf dem Stack gehalten werden. Allerdings führt dieser Ansatz zu mehr benötigter Rechenleistung.

b) *Heuristik durch Sortierung:* Da das Ziel im Spiel meist die Maximierung des Wertes bei geringem Gewicht ist, wird die Item-Liste vor der Berechnung absteigend nach dem Goldwert sortiert. Dies sorgt dafür, dass wertvolle Gegenstände (wie Edelsteine oder verzauberte Waffen) zuerst geprüft werden, wodurch der Schwellenwert für einen SUCCEEDED-Status oft deutlich früher erreicht wird.

c) *Effizienzsteigerung durch Multithreading:* Um die Rechenlast der Brute-Force-Suche zu bewältigen, wird die Berechnung über einen ExecutorService parallelisiert.

- Die Anzahl der Threads orientiert sich an den verfügbaren CPU-Kernen ($n - 1$).
- Jeder Thread startet seine Suche mit einem anderen Basis-Item aus der sortierten Liste.
- Terminierung: Sobald ein Thread eine gültige Beutekombination gefunden hat, signalisiert er dies über eine AtomicBoolean (solutionFound). Alle anderen Threads brechen ihre Suche daraufhin unmittelbar ab, um CPU-Ressourcen zu schonen.

C. Validierung: Suche vs. Verifikation

Ein zentraler Punkt der Untersuchung ist der Effizienzunterschied zwischen der Suche nach einer Lösung und deren Überprüfung. Während die Brute-Force-Suche im schlimmsten Fall eine exponentielle Anzahl an Kombinationen prüfen muss, erfolgt die Verifikation der gefundenen Skyrim-Ausrüstung in linearer Zeit $O(2n)$. Die Implementierung misst beide Zeiten, um zu verdeutlichen, dass die Bestätigung einer „legalen“ Traglast nur einen Bruchteil der Rechenzeit der eigentlichen Suche beansprucht.

VIII. ERGEBNISSE

Anbei können die Ergebnisse zweier Testdurchläufe heruntergeladen werden: 

Verwendete Parameter: Gewicht 300; Wert 1000; Erhobenes Datenset des Papers.

Zu beobachten ist vor Allem, dass die Präsortierung einen erheblichen Unterschied auf die Anzahl der Durchläufe hat. Mit den gegebenen Werten ist es einfach, ein valides Assignment zu finden, durch einsammeln der Gegenstände im Spiel unter Beachtung der in II-A kann das gefundene Assignment validiert werden.

LITERATUR

- [1] „Skyrim:helgen keep,“ besucht am 13. Dez. 2025. Adresse: https://en.uesp.net/wiki/Skyrim:Helgen_Keep.
- [2] „Skyrim:Ausdauer,“ besucht am 17. Dez. 2025. Adresse: [https://elderscrolls.fandom.com/de/wiki/Ausdauer_\(Skyrim\)](https://elderscrolls.fandom.com/de/wiki/Ausdauer_(Skyrim)).
- [3] „Skyrim:attributes,“ besucht am 17. Dez. 2025. Adresse: <https://en.uesp.net/wiki/Skyrim:Attributes>.
- [4] „Record and field types,“ besucht am 20. Dez. 2025. Adresse: <https://deepwiki.com/TES5Edit/TES5Edit/2.3-record-and-field-types>.
- [5] „Skyrim:leveled lists,“ besucht am 17. Dez. 2025. Adresse: https://en.uesp.net/wiki/Skyrim%3ALeveled_Lists.
- [6] „Skyrim:potato,“ besucht am 20. Dez. 2025. Adresse: [https://elderscrolls.fandom.com/de/wiki/Kartoffel_\(Skyrim\)](https://elderscrolls.fandom.com/de/wiki/Kartoffel_(Skyrim)).
- [7] *Knapsack problem*, in *Wikipedia*, Page Version ID: 1317349218, 17. Okt. 2025. besucht am 13. Dez. 2025. Adresse: https://en.wikipedia.org/w/index.php?title=Knapsack_problem&oldid=1317349218.
- [8] Univ.-Prof. Dr. Richard Kueng, MSc ETH, *Berechenbarkeit und Komplexität*, 2025. besucht am 13. Dez. 2025.
- [9] Eva Tardos, *Reduction from 3 SAT to subset sum*, März 2025. besucht am 13. Dez. 2025. Adresse: <https://www.cs.cornell.edu/courses/cs4820/2015sp/notes/reduction-subsetsum.pdf>.
- [10] „Sseedit,“ besucht am 17. Dez. 2025. Adresse: <https://www.nexusmods.com/skyrimsp/164>.
- [11] silvio201, *silvio201/SkyrimKnapsack*, original-date: 2025-12-14T20:02:22Z, 20. Dez. 2025. besucht am 20. Dez. 2025. Adresse: <https://github.com/silvio201/SkyrimKnapsack>.